

Entrega 5 - Dynamic Testing

Ángel David Suescun Romero

asuescunr@unal.edu.co

Laura Camila Perilla Torres

lperillat@unal.edu.co

Juan Pablo Rodríguez Briceño

jrodirguezbr@unal.edu.co

Wilson Franco Martinez

wfrancom@unal.edu.co

Asignatura: Ingeniería de software I

Docente: Oscar Eduardo Alvarez Rodriguez

Universidad Nacional de Colombia

Sede Bogotá

Bogotá D.C

Colombia

TABLA DE CONTENIDOS

Herramientas Utilizadas.....	3
Desactivación de usuarios (UserDeactivateService) - Angel Suescun.....	3
Validación de Stock antes de venta(SaleService) - Wilson Franco.....	7
Validación de Producto (ProductCreateService) - Angel Suescun.....	11
Actualización de Producto (ProductUpdateService) - Laura Camila Perilla Torres.....	14

ÍNDICE DE FIGURAS

Figura 1: Desactivación exitosa de un usuario.....	4
Figura 2: Desactivación exitosa de un usuario.....	5
Figura 3: Desactivación exitosa de un usuario.....	5
Figura 4: Desactivación exitosa de un usuario.....	6
Figura 5: Cobertura obtenida.....	7
Figura 6: Test Exitoso.....	7
Figura 7 : creación de cotización de venta de producto con stock suficiente.....	8
Figura 8: Rechazo de petición por stock insuficiente	9
Figura 9 : Previene que las ventas realizadas se modifiquen.....	10
Figura 10: cobertura obtenida.....	11
Figura 11: Texts Exitosos.....	11
Figura 12: Prueba para creación de producto.....	12
Figura 13: Prueba para producto ya existente.....	12
Figura 14: Prueba para proveedor no existente.....	13
Figura 15: Resultados de JaCoCo.....	13
Figura 16: Test Exitoso.....	13
Figura 17: Actualización exitosa de un producto.....	14
Figura 18: Conservación de valores existentes cuando algunos campos son nulos.....	16
Figura 19: Prueba para producto no encontrado.....	16
Figura 20: Cobertura obtenida.....	17
Figura 21: Test exitoso.....	17

Herramientas Utilizadas

- JUnit 5
- Mockito
- Maven
- JaCoCo

Desactivación de usuarios (UserDeactivateService) - Angel Suescun

Caso 1: Desactivación exitosa de un usuario

Objetivo: Verificar que un usuario existente pueda ser desactivado correctamente por un usuario con los permisos adecuados.

Resultado esperado: El usuario es desactivado y enviado al repositorio para su persistencia.

Caso límite cubierto: Verificación de que el usuario objetivo sea diferente del usuario autenticado, evitando falsos positivos en la validación de auto-desactivación.

```

@Test
public void shouldDeactivateUserSuccessfully() {

    User user = new UserBuilder()
        .setId(2)
        .setName("Test User")
        .setEmail("test@gmail.com")
        .setState(new ActiveState())
        .setRole(new Role( id: 1, name: "ADMIN_EMPRESA"))
        .build();

    when(userReadRepository.findById(user.getId())).thenReturn(Optional.of(user));

    when(activeSessionService.getActiveToken()).thenReturn( t: "token");

    when(jwtService.extractUserId( token: "token")).thenReturn( t: 1);

    when(userWriteRepository.save(any(User.class)))
        .thenAnswer( InvocationOnMock invocation -> invocation.getArgument( t: 0));

    User result = userDeactivateService.deactivateUser(user.getId());

    assertNotNull(result);

    verify(userWriteRepository).save(user);
}

```

Figura 1: Desactivación exitosa de un usuario

Caso 2: Usuario no encontrado

Objetivo: Verificar el comportamiento del sistema cuando se intenta desactivar un usuario inexistente.

Resultado esperado: El sistema lanza una excepción (NoSuchElementException).

Caso límite cubierto: Identificador de usuario inexistente o no registrado en el sistema.

```

@Test
public void shouldThrowExceptionWhenUserNotFound() {

    when(userReadRepository.findById(999)).thenReturn( Optional.empty());

    assertThrows(NoSuchElementException.class, () -> userDeactivateService.deactivateUser( userId: 999));
}

```

Figura 2: Desactivación exitosa de un usuario

Caso 3: Usuario ya inactivo

Objetivo: Verificar que el sistema impida desactivar nuevamente un usuario que ya se encuentra inactivo.

Resultado esperado: El sistema lanza una excepción (IllegalStateException).

Caso límite cubierto: Intento de aplicar una transición de estado inválida sobre un usuario previamente desactivado.

```

@Test
public void shouldThrowExceptionWhenUserIsAlreadyInactive() {

    User user = new UserBuilder()
        .setId(2)
        .setName("Test User")
        .setEmail("test@gmail.com")
        .setState(new InactiveState())
        .setRole(new Role( id: 1, name: "ADMIN_EMPRESA"))
        .build();

    when(userReadRepository.findById(user.getId())).thenReturn(Optional.of(user));

    when(activeSessionService.getActiveToken()).thenReturn( Optional.of("token"));

    when(jwtService.extractUserId( token: "token")).thenReturn( Optional.of(1));

    assertThrows(IllegalStateException.class, () -> userDeactivateService.deactivateUser(user.getId()));
}

```

Figura 3: Desactivación exitosa de un usuario

Caso 4: Intento de auto-desactivación

Objetivo: Verificar que un usuario no pueda desactivar su propia cuenta mientras se encuentra autenticado.

Resultado esperado: El sistema lanza una excepción (IllegalArgumentException).

Caso límite cubierto: El identificador del usuario objetivo coincide exactamente con el identificador del usuario autenticado.

```
@Test
public void shouldThrowExceptionWhenTryingToDeactivateCurrentUser() {

    User user = new UserBuilder()
        .setId(1)
        .setName("Current User")
        .setEmail("current@gmail.com")
        .setState(new ActiveState())
        .setRole(new Role( id: 1, name: "ADMIN_EMPRESA"))
        .build();

    when(userReadRepository.findById(user.getId())).thenReturn(Optional.of(user));

    when(activeSessionService.getActiveToken()).thenReturn( t: "token");

    when(jwtService.extractUserId( token: "token")).thenReturn(user.getId());

    assertThrows(IllegalArgumentException.class, () -> userDeactivateService.deactivateUser(user.getId()));
}
```

Figura 4: Desactivación exitosa de un usuario

Cobertura obtenida:

Se utilizó JaCoCo para medir la cobertura de las pruebas implementadas.

- Cobertura de líneas: **100%**
- Cobertura de métodos: **100%**

UserDeactivateService

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
deactivateUser(Integer)		100%		n/a	0	1	0	6	0	1
UserDeactivateService(UserReadRepository, UserWriteRepository, AuthorizationService, JwtService, ActiveSessionService)		100%		n/a	0	1	0	7	0	1
validateNotCurrentUser(User)		100%		100%	0	2	0	4	0	1
lambda\$deactivateUser\$0()		100%		n/a	0	1	0	1	0	1
Total	0 of 64	100%	0 of 2	100%	0	5	0	17	0	4

Created with JaCoCo 0.8.

Figura 5: Cobertura obtenida

```
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.775 s -- in com.unerp.service.user.UserDeactivateServiceTest
[INFO] Results:
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco:0.8.13:report (report) @ unerp ---
[INFO] Loading execution data file C:\Users\Familia_Suescun\Desktop\Semestre 2026-2\IngeSoft-I\project-backend\target\jacoco.exec
[INFO] Analyzed bundle '' with 44 classes
[WARNING] Classes in bundle '' do not match with execution data. For report generation the same class files must be used as at runtime.
[WARNING] Execution data for class com/unerp/service/user/UserCreateService does not match.
[INFO] BUILD SUCCESS
[INFO] Total time: 7.205 s
[INFO] Finished at: 2026-06-13T17:50:36-05:00
[INFO]
```

Figura 6: Test Exitoso

Validación de Stock antes de venta(SaleService) - Wilson Franco

Caso 1: Creación exitosa de cotización con stock suficiente.

Objetivo: Verificar que un empleado pueda crear una cotización de venta cuando hay stock suficiente de los productos.

Resultado esperado: La cotización se crea exitosamente , el stock disminuye correctamente y se calcula el total de la venta.

Caso límite cubierto: Verificación de que la cantidad solicitada no supere el stock disponible, evitando falsos positivos en la validación del inventario.

```

@Test
public void shouldCreateSaleWhenStockIsSufficient() {

    Product product = new Product();
    product.setId(id: 1);
    product.setName(name: "Laptop Gamer");
    product.setStock(stock: 10);
    product.setPrice(new BigDecimal(val: "2500000"));

    SaleProduct saleProduct = new SaleProduct();
    saleProduct.setProductId(productId: 1);
    saleProduct.setQuantity(quantity: 5);
    saleProduct.setUnitPrice(new BigDecimal(val: "2450000"));

    Sale sale = new Sale();
    sale.setCustomerId(customerId: 1);
    sale.setStatus(SaleStatus.por_enviar);
    sale.setProducts(List.of(saleProduct));

    when(productService.getProductById(id: 1)).thenReturn(product);
    when(saleRepository.save(any(Sale.class))).thenAnswer(invocation -> invocation.getArgument(0));

    Sale result = saleService.createSale(sale);

    assertNotNull(result);
    verify(productService, times(1)).updateStock(eq(1), eq(-5));
    verify(saleRepository, times(1)).save(any(Sale.class));
}

```

Figura 7 : creación de cotización de venta de producto con stock suficiente.

Caso 2:Rechazo de cotización por stock insuficiente.

Objetivo: Verificar que el sistema rechace la creación de una cotización cuando el stock disponible es menor a la cantidad solicitada.

Resultado esperado: El sistema lanza una excepción `IllegalStateException` y no permite la creación de la cotización.

Caso límite cubierto: Intento de vender más unidades de las disponibles en inventario, previniendo ventas sin disponibilidad de productos.


```

@Test
public void shouldRejectSaleWhenStockIsInsufficient() {

    Product product = new Product();
    product.setId(id: 1);
    product.setName(name: "Laptop Gamer");
    product.setStock(stock: 10);
    product.setPrice(new BigDecimal(val: "2500000"));

    SaleProduct saleProduct = new SaleProduct();
    saleProduct.setProductId(productId: 1);
    saleProduct.setQuantity(quantity: 15);
    saleProduct.setUnitPrice(new BigDecimal(val: "2450000"));

    Sale sale = new Sale();
    sale.setCustomerId(customerId: 1);
    sale.setStatus(SaleStatus.por_enviar);
    sale.setProducts(List.of(saleProduct));

    when(productService.getProductById(id: 1)).thenReturn(product);

    IllegalStateException exception = assertThrows(IllegalStateException.class, 0 -> {
        | saleService.createSale(sale);
    });

    assertTrue(exception.getMessage().contains(s: "Stock insuficiente"));
    verify(saleRepository, never()).save(any(Sale.class));
}

```

Figura 8: Rechazo de petición por stock insuficiente .

Caso 3: Prevención de modificación de venta entregada.

Objetivo: verificar que el sistema impida modificar una venta que ya ha sido entregada.

Resultado esperado: El sistema lanza una excepción `IllegalStateException` y no permite modificar la venta.

Caso límite cubierto: intento de aplicar una transición de estado inválida sobre una venta previamente entregada (entregado -> cualquier otro estado)

```

@Test
public void shouldRejectSaleWhenStockIsZero() {

    Product product = new Product();
    product.setId(id: 2);
    product.setName(name: "Producto Sin Stock");
    product.setStock(stock: 0);
    product.setPrice(new BigDecimal(val: "50000"));

    SaleProduct saleProduct = new SaleProduct();
    saleProduct.setProductId(productId: 2);
    saleProduct.setQuantity(quantity: 1);
    saleProduct.setUnitPrice(new BigDecimal(val: "50000"));

    Sale sale = new Sale();
    sale.setCustomerId(customerId: 1);
    sale.setStatus(SaleStatus.por_enviar);
    sale.setProducts(List.of(saleProduct));

    when(productService.getProductById(id: 2)).thenReturn(product);

    IllegalStateException exception = assertThrows(IllegalStateException.class, () -> {
        saleService.createSale(sale);
    });

    assertTrue(exception.getMessage().contains(s: "Stock insuficiente"));
    verify(saleRepository, never()).save(any(Sale.class));
}

```

Figura 9 : Previene que las ventas realizadas se modifiquen.

Cobertura obtenida:

Se utilizó JaCoCo para medir la cobertura de las pruebas implementadas.

- Cobertura de líneas: **100%**
- Cobertura de métodos: **100%**

createSale(Sale)		100%		100%	0	4	0	15	0	1	
SaleService(SaleRepository, ProductService)		100%		n/a	0	1	0	4	0	1	
lambda\$createSale\$0(SaleProduct)		100%		n/a	0	1	0	1	0	1	
Total		107 of 200	46%	8 of 14	42%	11	17	21	40	7	10

Figura 10: cobertura obtenida.

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 14.77 s -- in com.unerp.UnerpApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 25.451 s
[INFO] Finished at: 2026-06-14T21:14:47-05:00
[INFO] -----
```

Figura 11: Texts Exitosos.

Validación de Producto (ProductCreateService) - Angel Suescun

Caso 1: Creación exitosa de un Product

Objetivo: Verificar que el sistema acepte la creación de un producto cuando no hay un mismo producto creado y existe un proveedor

Resultado esperado: El producto es creado y es enviado al repositorio para su permanencia.

Caso límite cubierto: Verificación de que no haya un producto creado con nombre y lote repetidos y que haya un proveedor.

```
@Test
public void shouldCreateProduct() {

    doNothing().when(authorizationService).validatePermission(any());

    when(supplierReadRepository.existsById(any())).thenReturn(true);

    when(productWriteRepository.save(any(Product.class))).thenAnswer(invocation
-> invocation.getArgument(0));

    Product product = productCreateService.createProduct("Papas", "Deliciosas
papas", 10, Double.valueOf("2500"), "PapasBatch", null, 1);

    assertNotNull(product);
    assertEquals("Papas", product.getName());
    assertEquals("Deliciosas papas", product.getDescription());
    assertEquals(10, product.getStock());
    assertEquals(Double.valueOf("2500"), product.getPrice());
    assertEquals("PapasBatch", product.getBatch());
```

```

    assertEquals(1, product.getSupplierId());
    verify(productWriteRepository).save(any(Product.class));
}

```

Figura 12: Prueba para creación de producto.

Caso 2: Product con nombre y lote ya existe

Objetivo: Verificar que el sistema rechace la creación de un producto cuando otro producto con el mismo nombre y lote ya existen

Resultado esperado: El sistema lanza una excepción `IllegalStateException` y no permite la creación del producto.

Caso límite cubierto: Intento de creación de producto, previniendo productos repetidos.

```

@Test
public void shouldThrowExceptionWhenProductAlreadyExists() {

    doNothing().when(authorizationService).validatePermission(any());
    when(productReadRepository.existsByName("Papas")).thenReturn(true);
    when(productReadRepository.existsByBatch("PapasBatch")).thenReturn(true);

    assertThrows(
        IllegalArgumentException.class,
        () ->
            productCreateService.createProduct("Papas", "Deliciosas papas", 10,
        Double.valueOf("2500"), "PapasBatch", null, 1));
}

```

Figura 13: Prueba para producto ya existente.

Caso 3: Supplier de producto no existe

Objetivo: Verificar que el sistema rechace la creación de un producto cuando el proveedor no existe

Resultado esperado: El sistema lanza una excepción `IllegalStateException` y no permite la creación del producto.

Caso límite cubierto: Intento de crear un producto de un proveedor inexistente, asegurando que no hayan productos de los cuales no se tiene información del proveedor.

```

@Test
public void shouldThrowExceptionWhenSupplierDoesNotExist() {

    doNothing().when(authorizationService).validatePermission(any());
    when(productReadRepository.existsByName("Papas")).thenReturn(true);
    when(productReadRepository.existsByBatch("PapasBatch")).thenReturn(true);
    lenient().when(supplierReadRepository.existsById(1)).thenReturn(false);

    assertThrows(
        IllegalArgumentException.class,
        () ->
            productCreateService.createProduct("Papas", "Deliciosas papas", 10,
            Double.valueOf("2500"), "PapasBatch", null, 1));
}

```

Figura 14: Prueba para proveedor no existente

Se utilizó JaCoCo para medir la cobertura de las pruebas implementadas.

- Cobertura de líneas: **100%**
- Cobertura de métodos: **100%**

com.unerp.service.product > ProductCreateService

ProductCreateService

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
createProduct(String, String, Integer, Double, String, LocalDate, Integer)		100%	n/a		0	1	0	13	0	1
ProductCreateService(ProductReadRepository, SupplierReadRepository, ProductWriteRepository, AuthorizationService)		100%	n/a		0	1	0	6	0	1
validateNameExists(String)		100%		100%	0	2	0	5	0	1
validateBatchExists(String)		100%		100%	0	2	0	5	0	1
validateProductExists(String, String)		100%		100%	0	3	0	3	0	1
validateSupplierExists(Integer)		100%		100%	0	2	0	3	0	1
Total	0 of 106	100%	0 of 10	100%	0	11	0	35	0	6

Created at: 2026-06-15T00:38:30-05:00

Figura 15: Resultados de JaCoCo

```

[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 3.525 s -- in com.unerp.service.product.ProductCreateServiceTest
[INFO] Results:
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- jacoco-maven-plugin:0.8.13:report (report) @ unerp ---
[INFO] Loading execution data file /home/jp/GitHubRepositories/IngSoft/project-backend/target/jacoco.exec
[INFO] Analyzed bundle '' with 46 classes
[INFO] --- maven-jar-plugin:3.4.2:jar (default-jar) @ unerp ---
[INFO] Building jar: /home/jp/GitHubRepositories/IngSoft/project-backend/target/unerp-0.0.1-SNAPSHOT.jar
[INFO] --- spring-boot-maven-plugin:4.0.6:repackage (repackage) @ unerp ---
[INFO] Replacing main artifact /home/jp/GitHubRepositories/IngSoft/project-backend/target/unerp-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to /home/jp/GitHubRepositories/IngSoft/project-backend/target/unerp-0.0.1-SNAPSHOT.jar.original
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 11.022 s
[INFO] Finished at: 2026-06-15T00:38:30-05:00
[INFO] -----

```

Figura 16: Test Exitoso

Actualización de Producto (ProductUpdateService) - Laura Camila Perilla Torres

Caso 1: Actualización exitosa de un producto

Objetivo: Verificar que el sistema permita actualizar correctamente un producto existente dentro del módulo de inventario, modificando sus datos principales como nombre, descripción, stock, precio, lote, fecha de vencimiento y proveedor.

Resultado esperado: El producto es actualizado correctamente con los nuevos valores suministrados y enviado al repositorio para su persistencia.

Caso límite cubierto: Verificación de que todos los campos editables del producto puedan ser reemplazados correctamente sin perder el identificador original del producto.

```
64         batch: "Lote-B",
65         LocalDate.of(year: 2027, month: 1, dayOfMonth: 15),
66         supplierId: 2);
67
68     assertNotNull(result);
69     assertEquals(1, result.getId());
70     assertEquals("Arroz Premium", result.getName());
71     assertEquals("Arroz blanco premium", result.getDescription());
72     assertEquals(50, result.getStock());
73     assertEquals(Double.valueOf("4200"), result.getPrice());
74     assertEquals("Lote-B", result.getBatch());
75     assertEquals(LocalDate.of(year: 2027, month: 1, dayOfMonth: 15), result.getExpirationDate());
76     assertEquals(2, result.getSupplierId());
77
78     verify(authorizationService).validatePermission(any());
79     verify(productWriteRepository).save(any(Product.class));
80 }
```

Figura 17: Actualización exitosa de un producto.

Caso 2: Conservación de valores existentes cuando algunos campos son nulos

Objetivo: Verificar que el sistema conserve los valores anteriores de un producto cuando los campos de actualización llegan como null.

Resultado esperado: El producto mantiene sus valores originales en los campos que no fueron enviados, evitando que información existente sea reemplazada accidentalmente por valores nulos.

Caso límite cubierto: Actualización parcial de producto, asegurando que datos como nombre, descripción, stock, precio, lote, fecha de vencimiento y proveedor se conserven cuando no se envían nuevos valores.

```
@Test
public void shouldKeepExistingValuesWhenUpdateFieldsAreNull() {
    Product existingProduct =
        new Product(
            id: 3,
            name: "Aceite",
            description: "Aceite vegetal",
            stock: 15,
            Double.valueOf(s: "12000"),
            batch: "Lote-C",
            LocalDate.of(year: 2026, month: 8, dayOfMonth: 20),
            supplierId: 4);

    doNothing().when(authorizationService).validatePermission(any());
    when(productReadRepository.findById(3)).thenReturn(Optional.of(existingProduct));
    when(productWriteRepository.save(any(Product.class)))
        .thenReturn(invocation -> invocation.getArgument(0));

    Product result =
        productService.updateProduct(
            id: 3,
            name: null,
            description: null,
            stock: 30,
            price: null,
            batch: null,
            expirationDate: null,
            supplierId: null);

    description: null,
    stock: 30,
    price: null,
    batch: null,
    expirationDate: null,
    supplierId: null);

    assertNotNull(result);
    assertEquals(3, result.getId());
    assertEquals("Aceite", result.getName());
    assertEquals("Aceite vegetal", result.getDescription());
    assertEquals(30, result.getStock());
    assertEquals(Double.valueOf(s: "12000"), result.getPrice());
    assertEquals("Lote-C", result.getBatch());
    assertEquals(LocalDate.of(year: 2026, month: 8, dayOfMonth: 20), result.getExpirationDate());
    assertEquals(4, result.getSupplierId());

    verify(authorizationService).validatePermission(any());
    verify(productWriteRepository).save(any(Product.class));
}
```

Figura 18: Conservación de valores existentes cuando algunos campos son nulos.

Caso 3: Producto no encontrado

Objetivo: Verificar que el sistema rechace la actualización de un producto que no existe dentro del inventario.

Resultado esperado: El sistema lanza una excepción `IllegalArgumentException` con el mensaje `Product doesn't exist` y no ejecuta ninguna operación de guardado.

Caso límite cubierto: Identificador de producto inexistente o no registrado en el sistema, evitando actualizaciones sobre registros inválidos.

```
@Test
public void shouldThrowExceptionWhenProductDoesNotExist() {
    doNothing().when(authorizationService).validatePermission(any());
    when(productReadRepository.findById(99)).thenReturn(Optional.empty());

    IllegalArgumentException exception =
        assertThrows(
            IllegalArgumentException.class,
            () ->
                productUpdateService.updateProduct(
                    id: 99,
                    name: "Producto inexistente",
                    description: "No existe",
                    stock: 10,
                    Double.valueOf(s: "5000"),
                    batch: "Lote-X",
                    LocalDate.of(year: 2026, month: 10, dayOfMonth: 10),
                    supplierId: 1));

    assertEquals("Product doesn't exist", exception.getMessage());

    verify(authorizationService).validatePermission(any());
    verify(productWriteRepository, never()).save(any(Product.class));
}
```

Figura 19: Prueba para producto no encontrado.

Cobertura obtenida:

Se utilizó JaCoCo para medir la cobertura de las pruebas implementadas.

- Cobertura de líneas: **100%**
- Cobertura de métodos: **100%**

ProductUpdateService

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Method
updateProduct(Integer,String,String,Integer,Double,String,LocalDate,Integer)		100%		100%	0	8	0
ProductUpdateService(ProductReadRepository,ProductWriteRepository,AuthorizationService)		100%		n/a	0	1	0
lambda\$updateProduct\$0()		100%		n/a	0	1	0
Total	0 of 105	100%	0 of 14	100%	0	10	0

Figura 20: Cobertura obtenida.

```
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.047 s -- in com.unerp.service.product.P
roductUpdateServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.13:report (report) @ unerp ---
[INFO] Loading execution data file C:\Users\camil\project-backend\target\jacoco.exec
[INFO] Analyzed bundle '' with 46 classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 6.002 s
[INFO] Finished at: 2026-06-16T10:52:14-05:00
[INFO]
```

Figura 21: Test exitoso.